

Présentation du projet "Synthétiseur"

2025-2026

Groupe BotWave :

Tinhinane AIT-MESSAOUD

Arthur ARNOLD

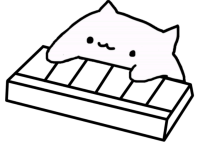
Paul CLUZEL

Noémie GIREAUD

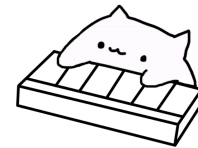
Client :

Emmanuel RIO

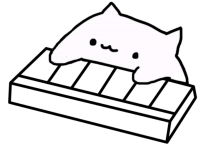
Sommaire



1. Introduction
2. Développement
 - 2.1 Conception de la solution
 - 2.2 Test et validation
 - 2.3 Description du travail effectué
3. Analyse critique
 - 3.1 Problèmes rencontrés
 - 3.2 Améliorations possibles
4. Conclusion

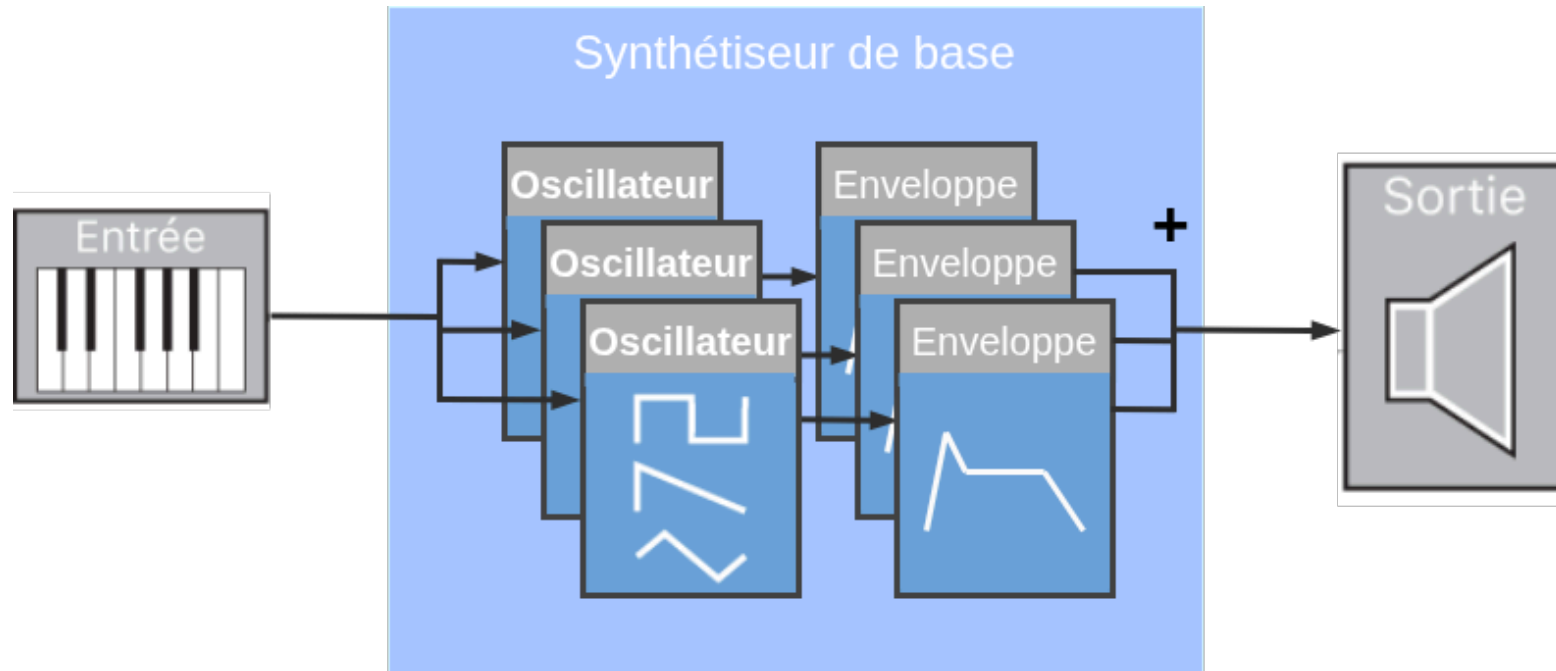


Introduction

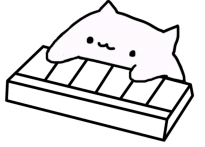


Qu'est-ce qu'un synthétiseur ?

- Création d'un outil de synthèse sonore



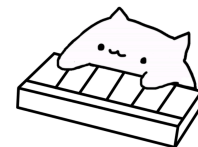
Description du problème



Objectif : Gérer la production d'un signal sonore paramétré depuis l'appui sur une / plusieurs touches jusqu'à sa sortie sur enceinte.

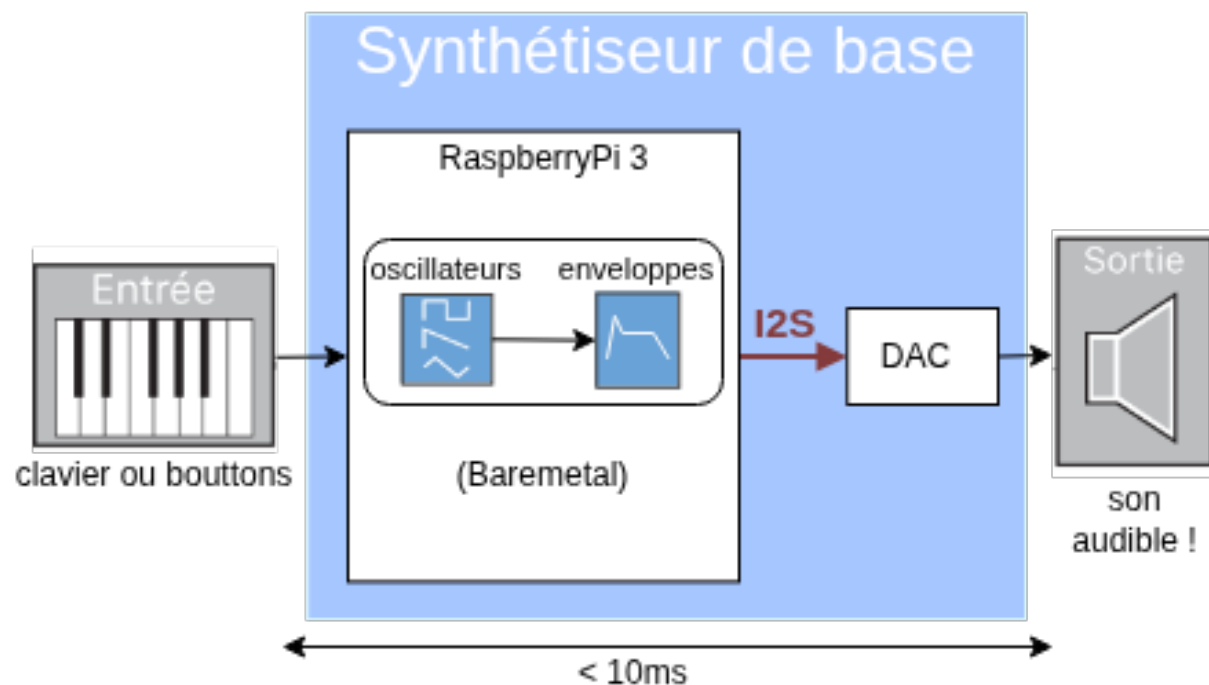
Plusieurs critères :

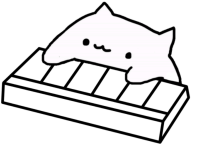
- Génération rapide du son (- de 10ms)
- Utilisation d'une carte hardware RaspberryPi
- Programmation baremetal
- Utilisation d'un DAC externe
- Utilisation du protocole de communication I2S entre la carte embarquée et le DAC



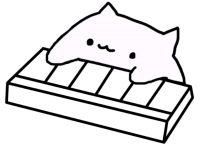
Différentes étapes :

- Lire une entrée de touches
- Générer les formes d'ondes (signal numérique)
- Les convertir en signal analogique
- Lire le signal analogique avec câble jack





Développement



2.1 Conception de la solution

2.1.1 Point de départ

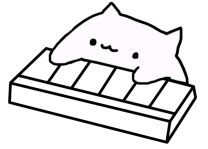
Materiel physique

- **inputs** : boutons, potentiomètres, clavier...
- **cartes** : RaspberryPi3 Model B ET STM32F407
- **transmission** : DAC (PCM5102)
- **output** : Enceinte (Jabra), casque, écouteurs ...

Baremetal !!

Modules et protocoles

- DMA : Direct Memory Access
- PCM : Pulse Code Modulation
- I2S : Inter-IC Sound



Réalisation de la solution en 2 plans parallèles

Raspberry Pi

Adaptation du kernel de Dr Poquet

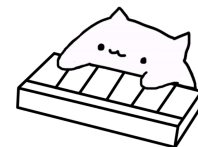
- Initialisation de la DMA et PCM
- Activation FPU
- Accès GPIOs
- Accès clavier avec registres à décalage
- Utilisation des horloges
- Intégration avec traitement du signal

Objectif final !

STM32F407

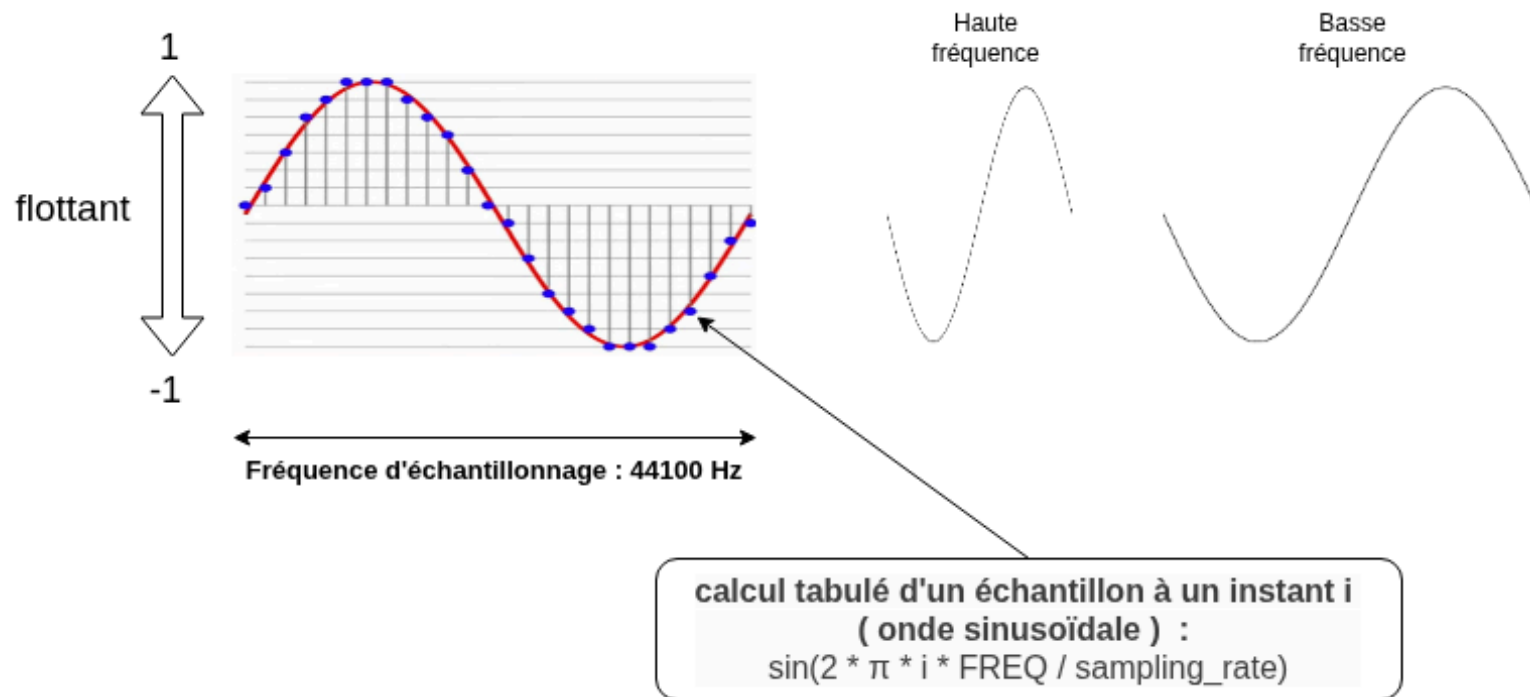
Focus sur le traitement de signal

- Découverte du buffer circulaire (DMA)
- Initialisation d'un tableau de donnée
- Création d'oscillateurs
- Mise en place d'enveloppes
- Réglage de l'amplitude

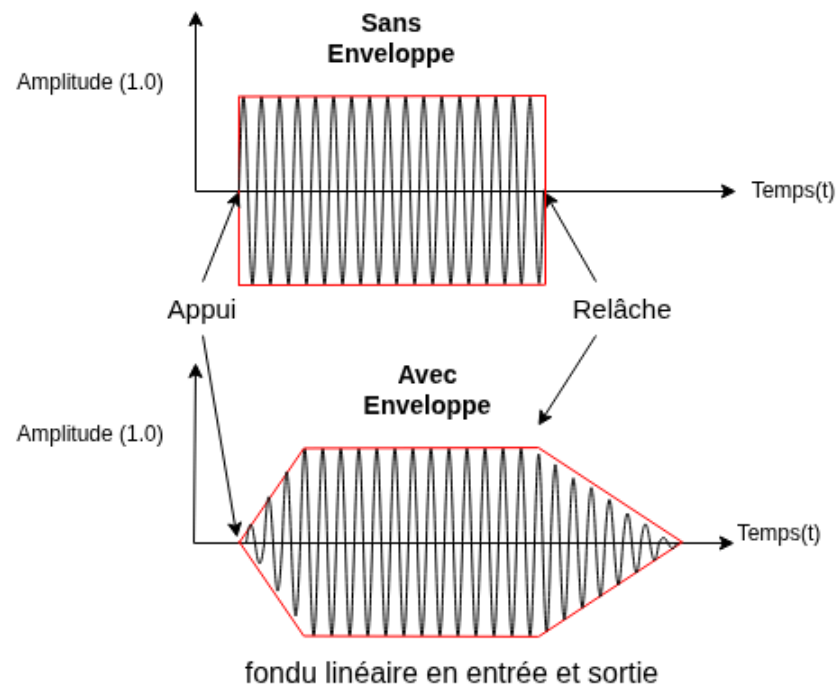
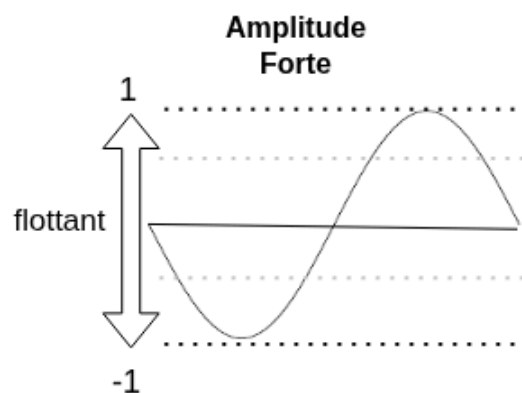
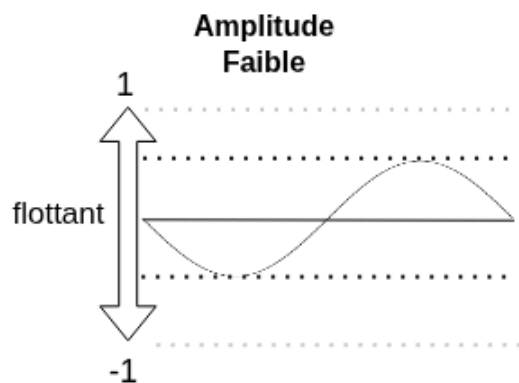
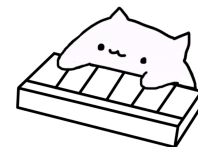


2.1.2 Traitement du signal

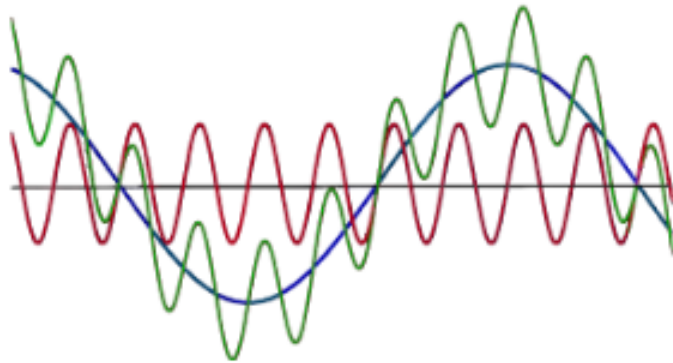
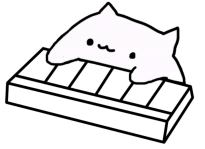
Création d'un oscillateur



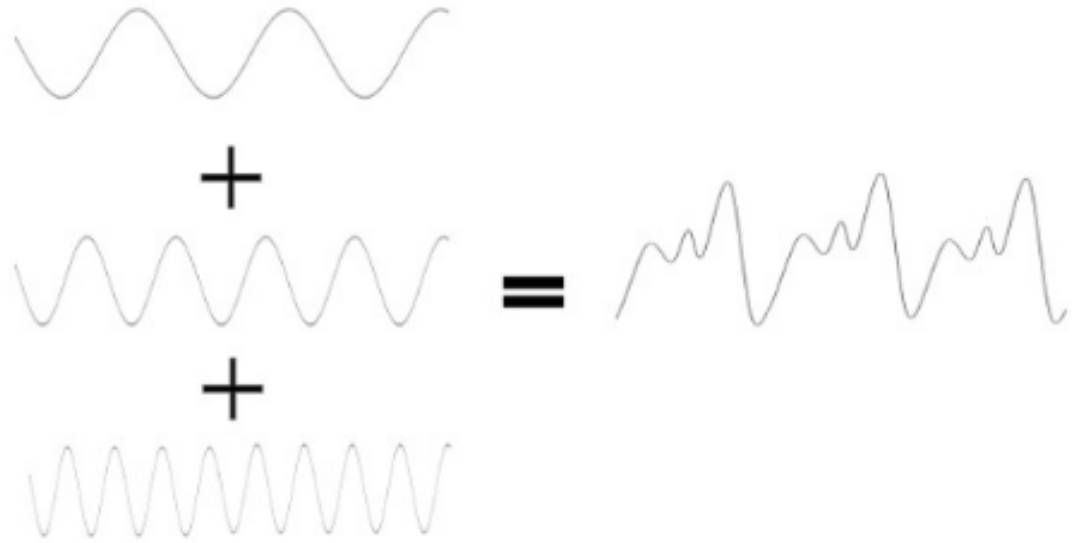
Amplitude et enveloppe



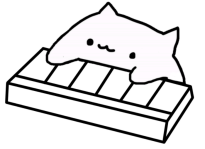
Addition d'ondes sonores



Amplitude augmentée !



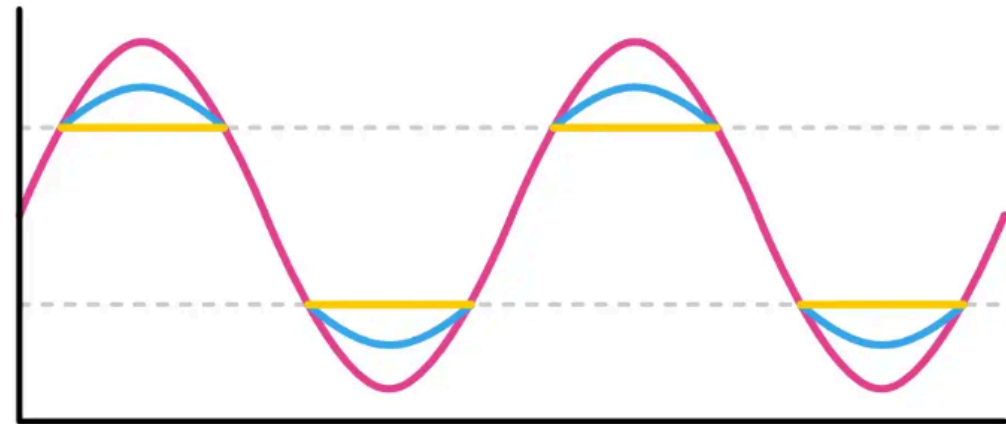
Soft Clipping



$$-\frac{2}{3}, \quad x \leq -1$$

$$x - \frac{x^3}{3}, \quad -1 \leq x \leq 1$$

$$\frac{2}{3}, \quad x \geq 1.$$



Normal



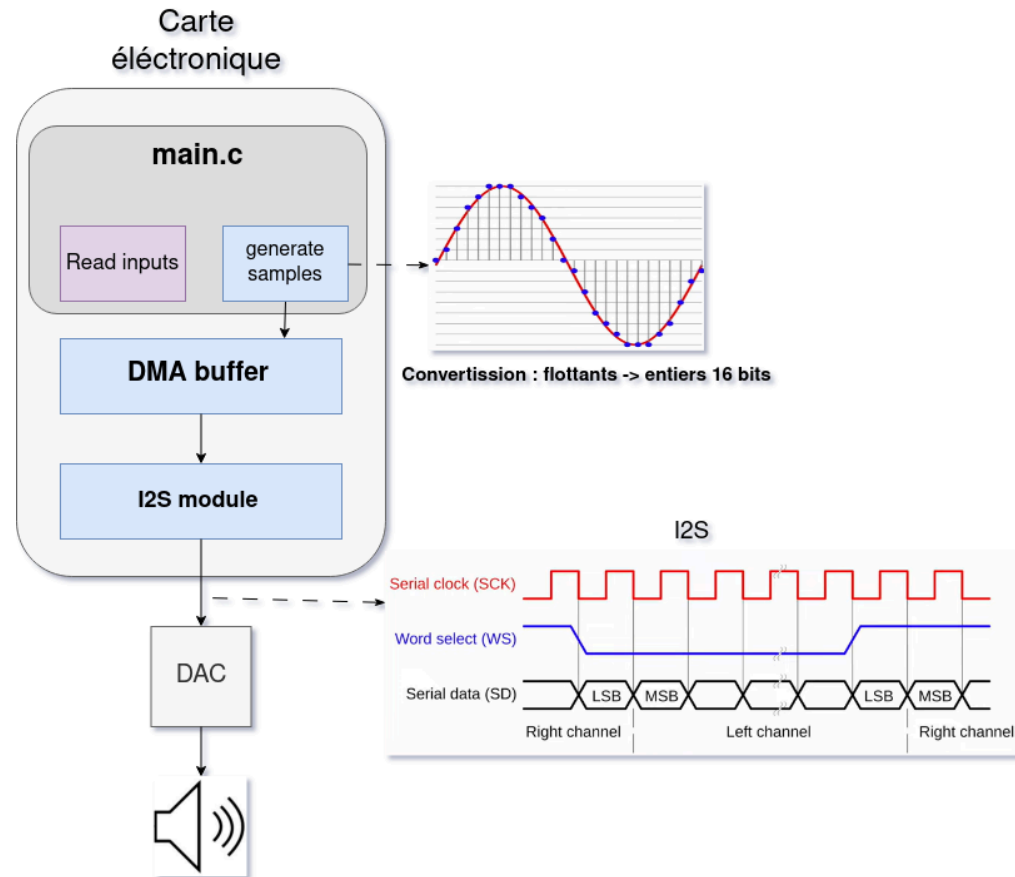
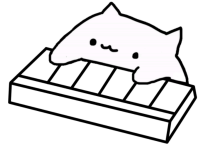
Hard Clipping



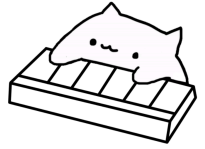
Soft Clipping



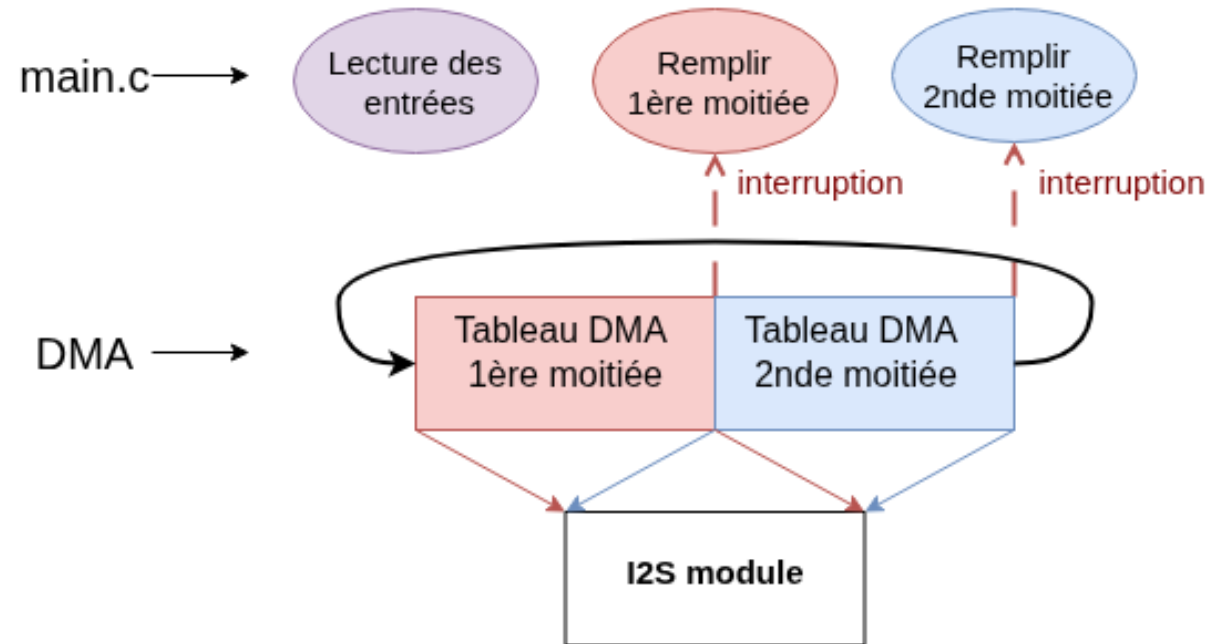
2.1.3 Transmission du signal



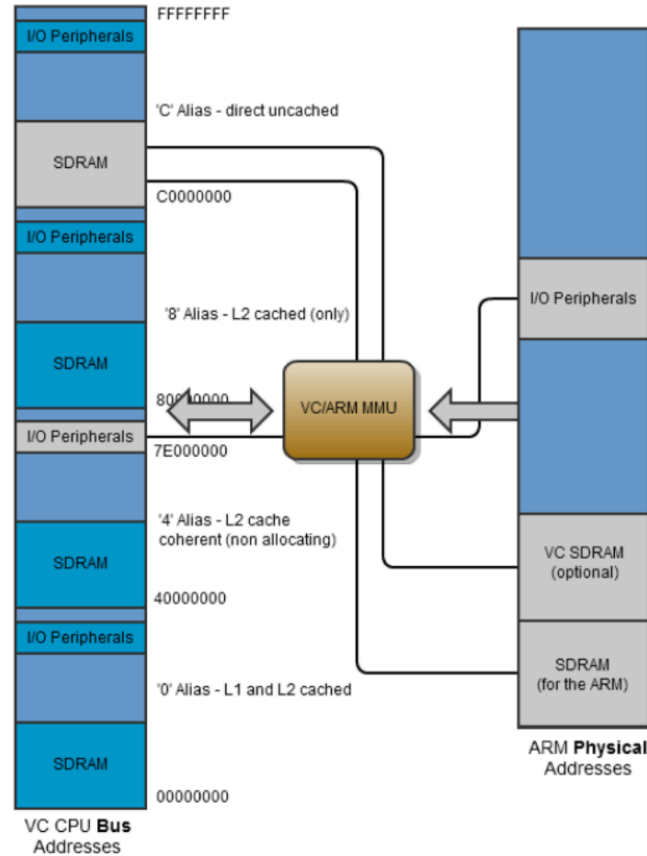
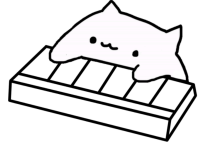
Utilisation de la DMA sur STM32F407

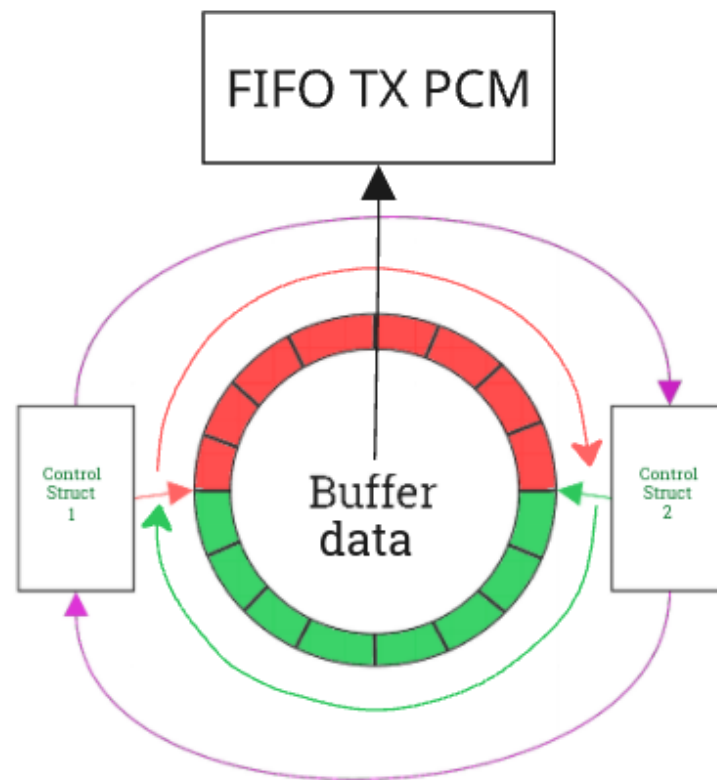
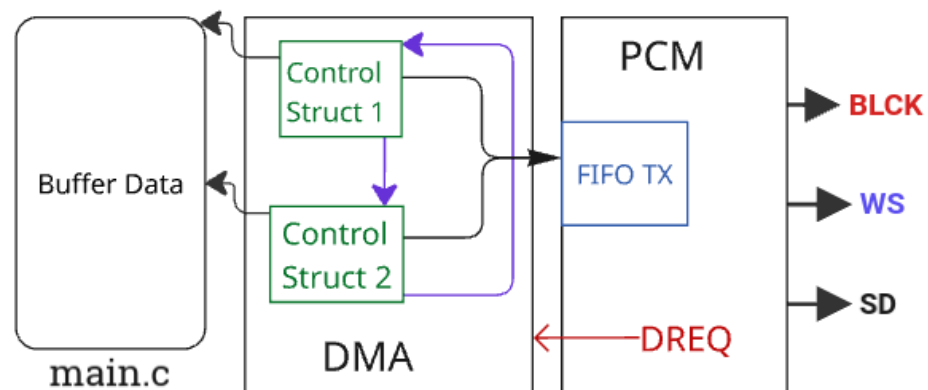
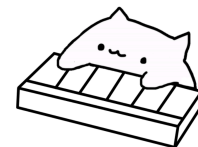


DMA avec tableau circulaire et interruptions

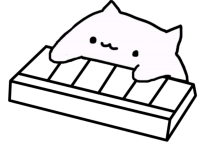


Utilisation de la DMA sur RaspberryPi 3

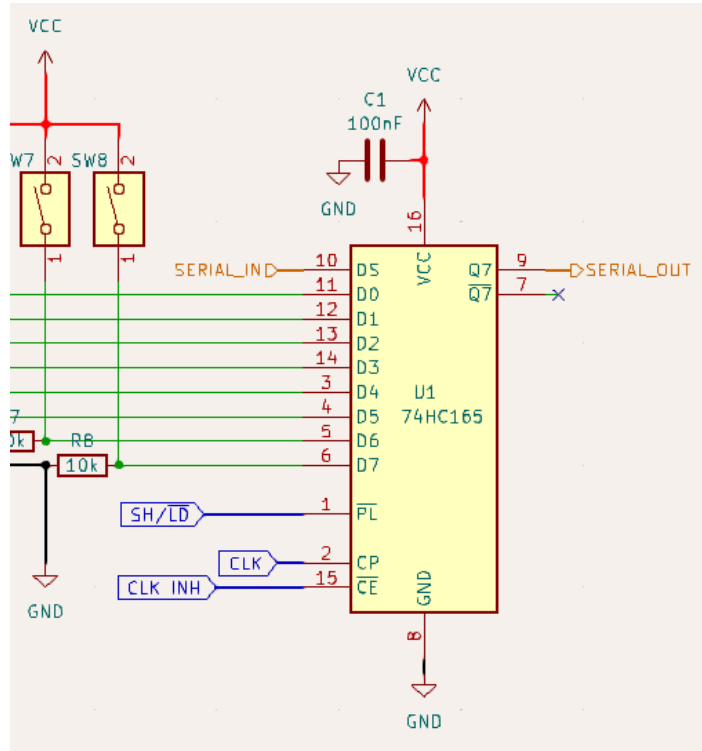




2.1.4 Traitement des entrées clavier



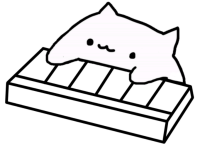
Utilisation d'un clavier avec 5 registres à décalages liés en **Daisy Chain**.



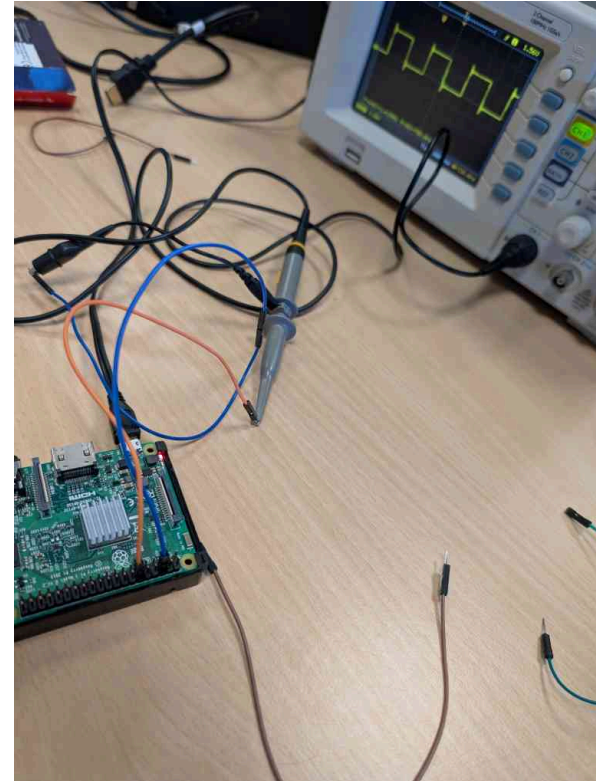
Étapes de lecture :

1. Placer SHIFT/LOAD à 0 afin de charger l'état parallèle des touches
2. Remplacer SHIFT/LOAD à 1 pour passer en mode décalage
3. Pour chaque touche (boucle de lecture) :
 - Lire la valeur de la sortie SERIAL
 - Générer une impulsion d'horloge (1 → 0) pour décaler le bit suivant
 - Répéter jusqu'à lecture complète des registres

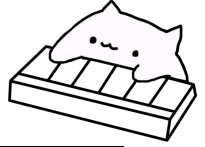
2.2 Tests et validation



- Tests incrémentaux.
- Analyse des fréquences via Oscilloscope et Audacity.
- Vérification grâce à l'oreille fine de Mr Rio

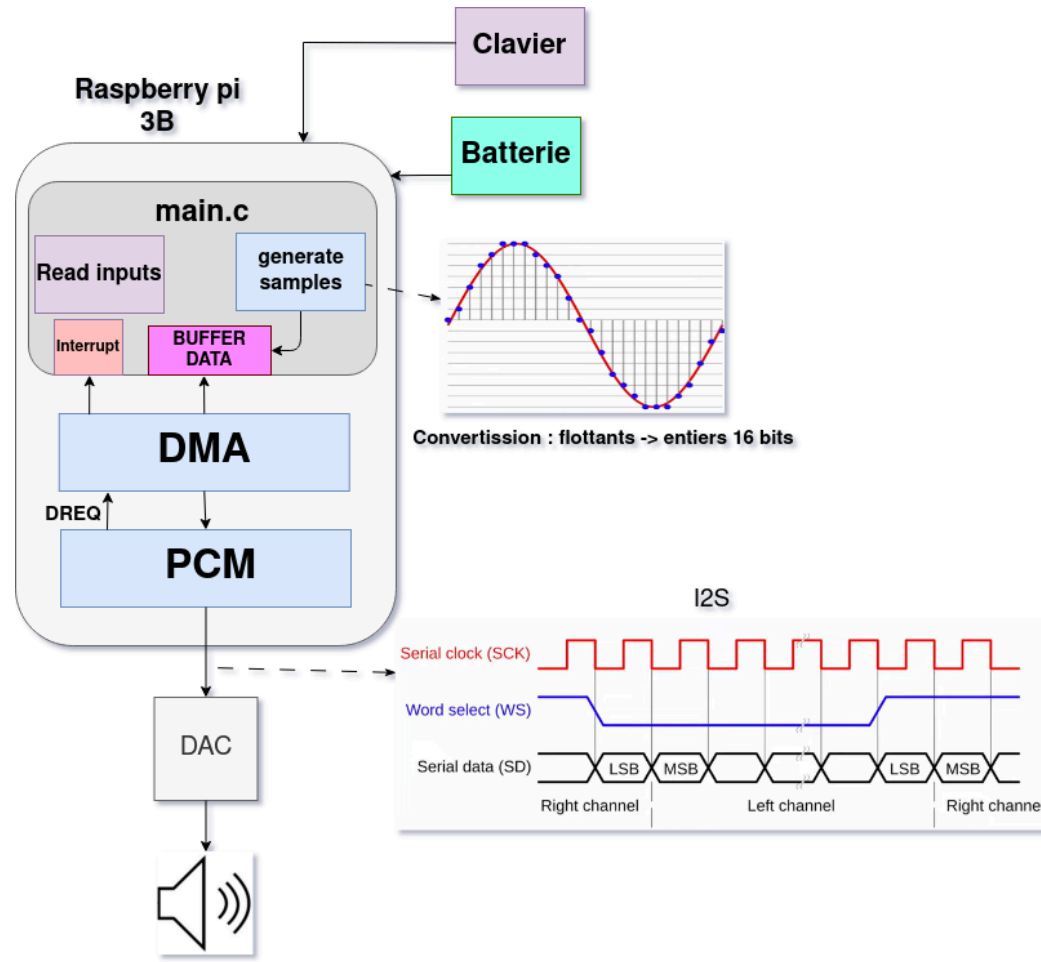
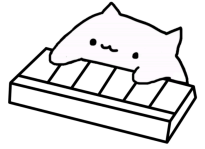


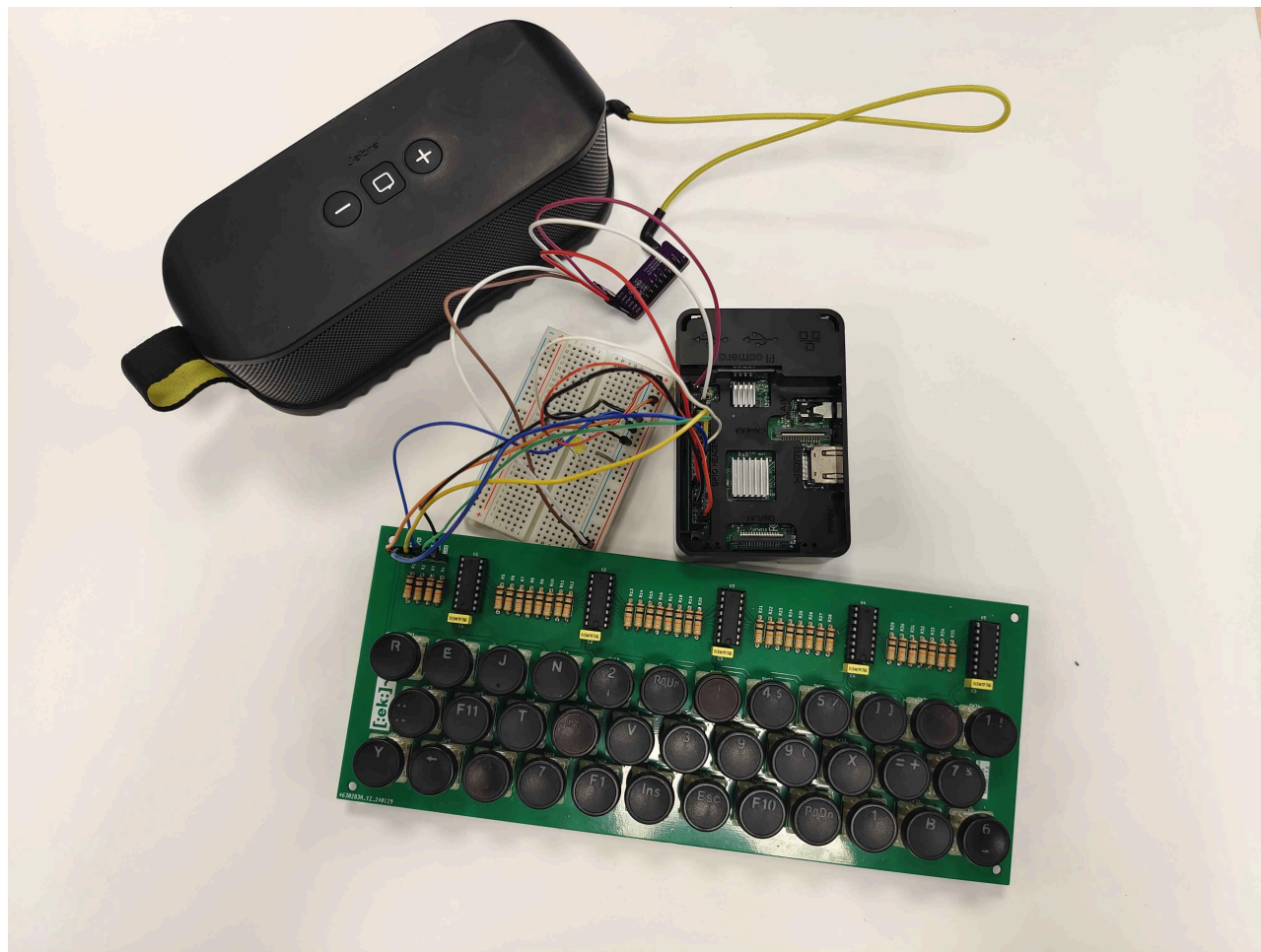
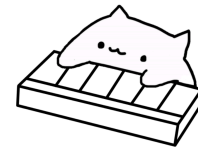
2.3 Description du travail effectué

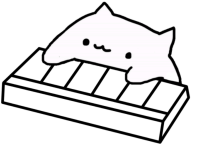


	STM32F407	Raspberry Pi
Protocole de communication	I2S via SPI	I2S via PCM -> Initialisation du module PCM
Utilisation de la DMA	DMA en mode circulaire avec des interruptions	DMA basée sur structure de contrôle Implémentation manuelle des interruptions
Gestion des touches	V1 : + 7 Boutons poussoirs pour notes + 2 Boutons pour changement d'octaves + Potentiomètre pour le volume V2 : Utilisation des 7 boutons du clavier	Mapping d'un tableau de fréquences sonores sur les 35 touches du clavier à un volume choisi préalablement
Processeur et vitesse d'exécution	Cortex™-M4 Vitesse : 168 MHz -O0 suffisant sans problème de timing	Quad-core 64 bits Vitesse : 1,4 GHz Nécessité de compiler en -O1 pour respecter les contraintes temporelles

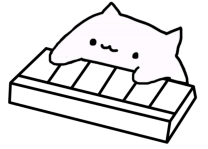
Documentation complète sur le dépôt github **Documentation**







Analyse critique

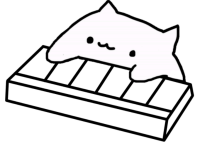


3.1 Problèmes rencontrés

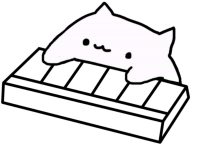
- Librairie Circle non fonctionnelle.
- Initialisation du composant PCM (Documentation incomplète)
- Initialisation des horloges de la raspberry pi pour le module PCM (Documentation incomplète)
- Problème de la masse d'EDF dû au bloc d'alimentation de la Raspberry Pi
- Utilisation d'adresses différentes pour faire fonctionner la DMA
- Intégration du traitement sonore vers Raspberry pi
- Certaines Pins GPIO inaccessibles sur STM32
- Exécution non conforme aux exigences temporelles en absence d'optimisations de compilation sur Raspberry Pi.
- Instabilité au niveau des connexions entraînant une dégradation de la sortie sonore avec apparition d'un bruit parasite.

Globalement : Traitement du signal très sensible !

3.2 Améliorations possibles

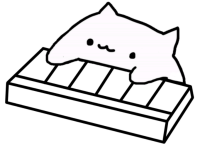


- Choix de la forme d'onde
- Form factor (Soudure & impression 3D)
- Utiliser une enveloppe ADSR
- Gestion du volume hors de la Raspberry Pi
- Choix des instruments de musique
- Intégration avec différent inputs (clavier piano par exemple)

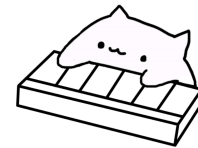


Conclusion

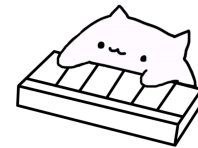
Conclusion



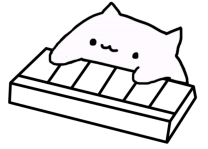
- Apprentissage des bases du traitement du signal
- Apprentissage de l'implémentation baremetal
- Découverte du protocole de communication I2S
- Découverte de l'utilisation de la DMA et des modules PCM / SPI
- Portage entre différentes cartes et adaptations aux difficultés



MERCI POUR VOTRE ATTENTION



Annexes



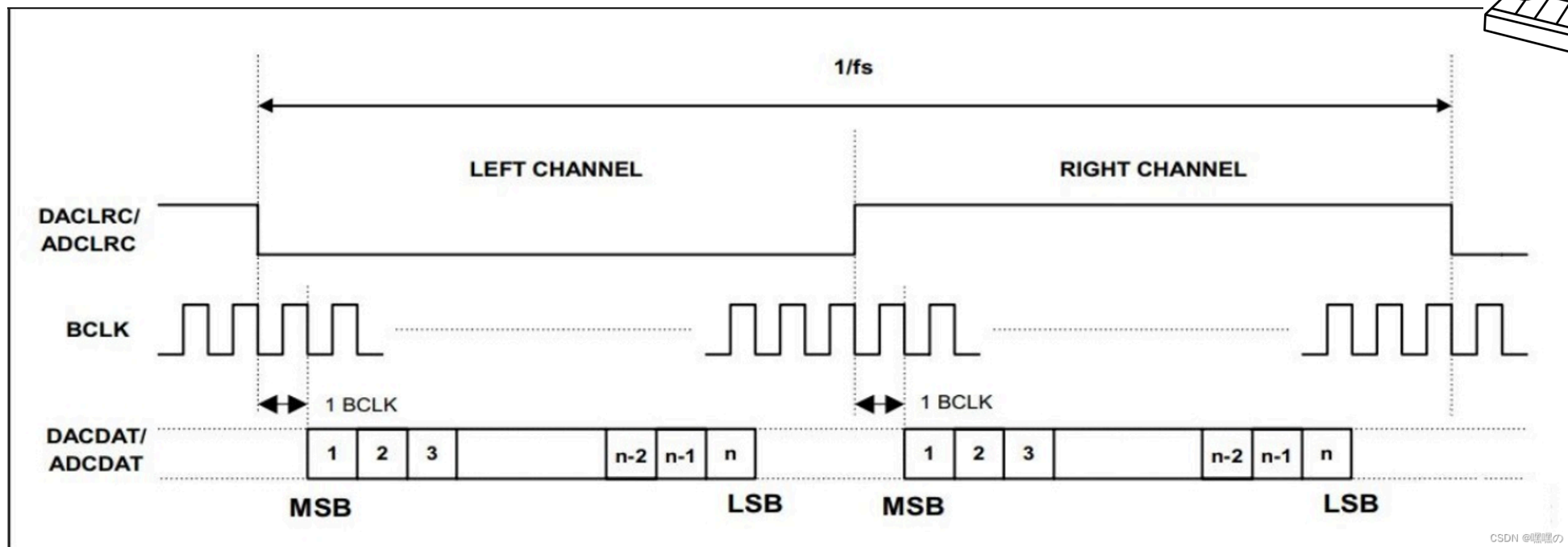
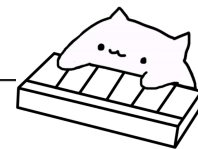
Le protocole I²S (Inter-IC Sound)

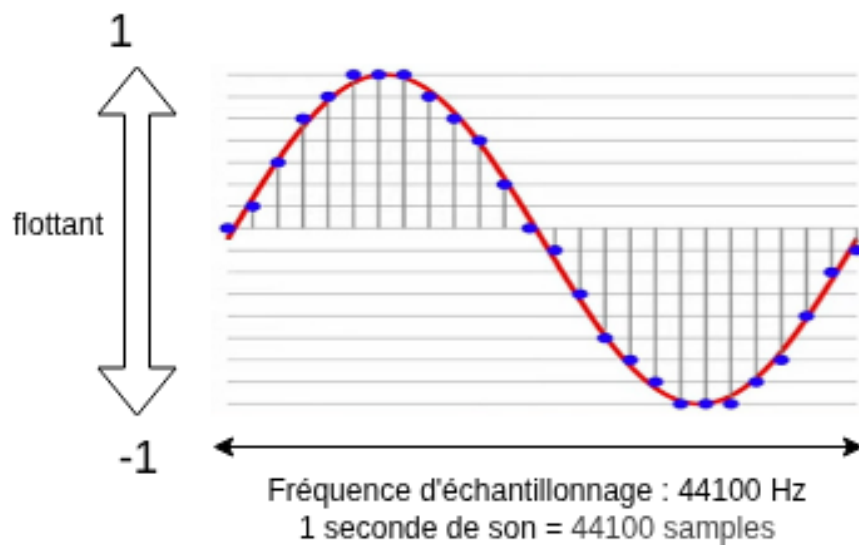
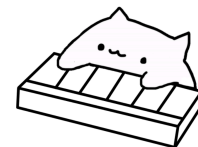
Usage : Standard de communication série synchrone dédié à l'audio numérique (entre processeur et DAC/ADC).

Architecture : Sépare les données d'horloge et les données série pour éliminer le jitter (scintillement du signal).

Pourquoi ?

- Gestion de deux canaux (Gauche/Droite) via un signal de sélection (WS).
- Protocoles classiques I²C et SPI non adapté pour transporter efficacement des données audio.
- Performance : Latence minimale, flux continu et déterministe.
- Qualité : Audio non compressé (PCM), préservant l'intégrité du signal original.

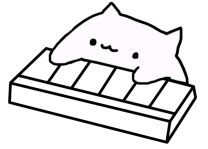




stéréo : on double chaque échantillon envoyé

sampling rate nb_bits / sample stéréo

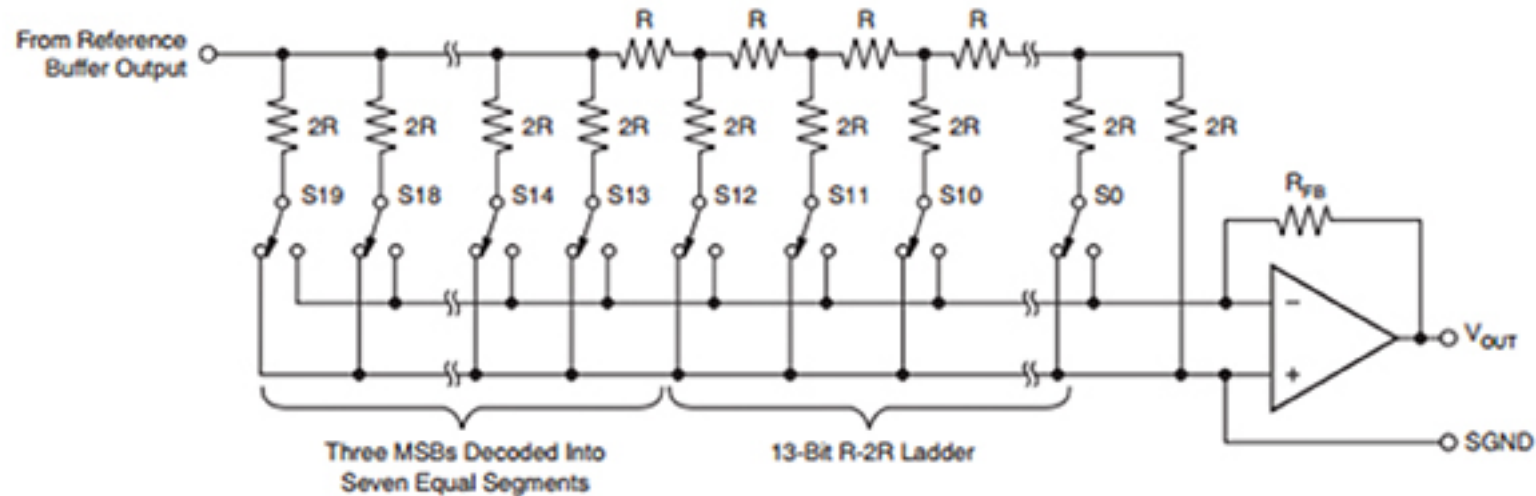
Fréquence de la clock I2S : $44100 * 16 * 2 = 1\,411\,200$ Hz

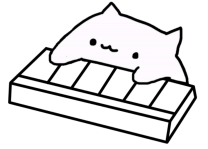


Le DAC (Digital to Analog Converter)

Un composant qui convertie un signal numérique en signal analogique.

Le composant reçoit un échantillon numérique qui activera ou non des résistances pour modifier la tension de sortie par rapport à sa tension d'entrée qu'il gardera pendant une période du signal LRCLK.





PCM (Pulse Code Modulation)

Le PCM est un composant permettant de représenter des signaux analogiques par des valeurs numériques. Il communique directement en I²S avec des fréquences spécifiques pour sélectionner le bon canal à utiliser, les bits à envoyer sur le bus.

Le composant est mappé en mémoire dans la raspberry pi par différent registres hardware et peut être utiliser de différentes manières :

- Interruption
- Polled
- DMA

